American International University – Bangladesh

Faculty of Science and Technology

Introduction to Data Science

Section- E, Group- C

Session: Fall 2025-26

Dataset Title

Professional Valorant Players Stats

Supervised by

KAMRUN NAHER KOLI

Group Member:

Name	ID
Arafat Hossain	23-50277-1
Symon Islam Parash	22-47340-2
Tohidul Islam Khan	22-49658-3

Final Term Project on Valorant Stats Dataset

1. Research Objectives

Research Problem: The competitive esports industry has many performance data yet identifying which performance data influences a player's overall rating most remains a challenge.

The main objective of this study is to:

- Analyze professional Valorant player performance using statistical match data.
- Identify key performance metrics that contribute to high player ratings.
- Build a predictive model to predict player rating and which in-game factors most influence player rating.

2. Data Collection (Web Scrapping)

Data Source: https://www.vlr.gg/stats

Data Collection Method-

- Web scraping was performed using R in RStudio
- Library used - rvest, stringr
- The data was collected from publicly available leaderboard pages.
- HTML tables containing player statistics were converted into structured data frame.

Web Scrapping Code:

```
> library(rvest)
> url<- "https://www.vlr.gg/stats"
> valorant <- read_html(url)
> library(stringr)
> player <- valorant %>%
+   html_elements(".text-of") %>%
+   html_text2()
> head(player)
[1] "cortezia"  "77even"    "Smoke"     "Chronicle" "Insanity"  "Nora"
> team <- valorant %>%
+   html_elements(".stats-player-country") %>%
+   html_text2()
> head(team)
[1] "SEN"  "FOCU" "YFP"  "VIT"  "RAD"  "PRS"
> RND <- valorant %>%
+   html_elements("td.mod-rnd") %>%
+   html_text2()
> head(RND)
[1] "351" "342" "372" "299" "230" "203"
> R <- valorant %>%
+   html_elements(".mod-rnd+ .mod-color-sq span") %>%
+   html_text2()
> head(R)
[1] "1.30" "1.29" "1.27" "1.26" "1.25" "1.24"
> ACS <- valorant %>%
+   html_elements(".mod-acs span") %>%
+   html_text2()
> head(ACS)
[1] "236.9" "277.3" "247.0" "234.5" "255.7" "228.0"
```

```
> KD <- valorant %>%
+   html_elements(".mod-acs+ .mod-color-sq span") %>%
+   html_text2()
> head(KD)
[1] "1.49" "1.40" "1.41" "1.46" "1.24" "1.32"
> KAST <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(7) span") %>%
+   html_text2()
> head(KAST)
[1] "76%" "70%" "74%" "76%" "76%" "80%"
> ADR <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(8) span") %>%
+   html_text2()
> head(ADR)
[1] "156.2" "174.9" "161.6" "151.5" "168.3" "147.2"
> KPR <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(9) span") %>%
+   html_text2()
> head(KPR)
[1] "0.86" "0.99" "0.89" "0.84" "0.90" "0.82"
> APR <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(10) span") %>%
+   html_text2()
> head(APR)
[1] "0.22" "0.14" "0.16" "0.22" "0.44" "0.37"
> FKPR <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(11) span") %>%
+   html_text2()
> head(FKPR)
[1] "0.10" "0.18" "0.15" "0.13" "0.11" "0.08"
> FDPR <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(12) span") %>%
+   html_text2()
> head(FDPR)
[1] "0.07" "0.13" "0.07" "0.10" "0.08" "0.08"
> HS <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(13) span") %>%
+   html_text2()
> head(HS)
[1] "34%" "41%" "21%" "25%" "33%" "30%"
> CL <- valorant %>%
+   html_elements(".mod-color-sq:nth-child(14) span") %>%
+   html_text2()
> head(CL)
[1] "14%" "14%" "14%" "19%" "10%" "21%"
> KMAX <- valorant %>%
+   html_elements(".mod-kmax a") %>%
+   html_text2()
> head(KMAX)
[1] "27" "34" "31" "30" "36" "22"
> K <- valorant %>%
+   html_elements(".mod-kmax+ td") %>%
+   html_text2()
> head(K)
[1] "303" "339" "331" "252" "207" "166"
> D <- valorant %>%
+   html_elements("td:nth-child(18)") %>%
+   html_text2()
> head(D)
[1] "203" "243" "234" "173" "167" "126"
> A <- valorant %>%
+   html_elements("td:nth-child(19)") %>%
+   html_text2()
> head(A)
[1] "77"  "49"  "61"  "65"  "102" "75"
> FK <- valorant %>%
+   html_elements("td:nth-child(20)") %>%
+   html_text2()
> head(FK)
[1] "35" "60" "54" "39" "25" "17"
> FD <- valorant %>%
+   html_elements("td:nth-child(21)") %>%
+   html_text2()
> head(FD)
[1] "24" "43" "27" "30" "18" "17"
```

```
> vlr_df <- data.frame(
+   Rank= 1:417,
+   Player_Name=player,
+   Team=team,
+   Round_Played=as.integer(RND),
+   Rating=as.numeric(R),
+   Avg_Combat_Score=as.numeric(ACS),
+   Kill_Death_Ratio=as.numeric(KD),
+   KAST_Percentage=as.numeric(gsub("%", "", KAST)),
+   Avg_Damage_Per_Round=as.numeric(ADR),
+   Kill_Per_Round=as.numeric(KPR),
+   Assist_Per_Round=as.numeric(APR),
+   FirstKill_Per_Round=as.numeric(FKPR),
+   FirstDeath_Per_Round=as.numeric(FDPR),
+   Headshot_Percentage=as.numeric(gsub("%", "", HS)),
+   Clutch_Percentage=as.numeric(gsub("%", "", CL)),
+   Max_Kill_Single_Map=as.integer(KMAX),
+   Kills=as.integer(K),
+   Deaths=as.integer(D),
+   Assists=as.integer(A),
+   First_Kills=as.integer(FK),
+   First_Deaths=as.integer(FD)
+ )
> head(vlr_df)
  Rank Player_Name Team Round_Played Rating Avg_Combat_Score Kill_Death_Ratio KAST_Percentage Avg_Damage_Per_Round Kill_Per_Round Assist_Per_Round
1    1    cortezia  SEN          351   1.30            236.9             1.49             76                156.2           0.86             0.22
2    2      77even FOCU          342   1.29            277.3             1.40             70                174.9           0.99             0.14
3    3       Smoke  YFP          372   1.27            247.0             1.41             74                161.6           0.89             0.16
4    4   Chronicle  VIT          299   1.26            234.5             1.46             76                151.5           0.84             0.22
5    5    Insanity  RAD          230   1.25            255.7             1.24             76                168.3           0.90             0.44
6    6        Nora  PRS          203   1.24            228.0             1.32             80                147.2           0.82             0.37
  FirstKill_Per_Round FirstDeath_Per_Round Headshot_Percentage Clutch_Percentage Max_Kill_Single_Map Kills Deaths Assists First_Kills First_Deaths
1                0.10                 0.07                  34               14                  27   303    203      77          35           24
2                0.18                 0.13                  41               14                  34   339    243      49          60           43
3                0.15                 0.07                  21               14                  31   331    234      61          54           27
4                0.13                 0.10                  25               19                  30   252    173      65          39           30
5                0.11                 0.08                  33               10                  36   207    167     102          25           18
6                0.08                 0.08                  30               21                  22   166    126      75          17           17
>
```

Collected Attributes:

Column Name	Description
Rank	Player ranking
Player_Name	In-game player name
Team	Team name
Round_Played	Total rounds played
Rating	Overall performance rating
Avg_Combat_Score	Average combat score
Kill_Death_Ratio	Kill to death ratio
KAST_Percentage	Percentage of rounds with Kill/Assist/Survive/Trade
Avg_Damage_Per_Round	Damage dealt per round
Kill_Per_Round	Average kills per round
Assist_Per_Round	Average assists per round
FirstKill_Per_Round	Opening kills per round

FirstDeath_Per_Round	Opening deaths per round
Headshot_Percentage	Percentage of headshots
Clutch_Percentage	Clutch success rate
Max_Kill_Single_Map	Highest kills in one map
Kills, Deaths, Assists, First_Kill, First_death	Total stats of Kills, Deaths, Assists, First_Kill, First_death

3. Data Understanding and Exploration

With the help of str() and summary() function, we can understand the structure of the dataset, including variables, data types and overall size. Also, confirms presence of numeric and categorical variables and get the minimum, maximum, mean and quartile values of each column.

```
> str(vlr_df)
'data.frame':	417 obs. of  21 variables:
 $ Rank               : int  1 2 3 4 5 6 7 8 9 10 ...
 $ Player_Name        : chr  "cortezia" "77even" "Smoke" "Chronicle" ...
 $ Team               : chr  "SEN" "FOCU" "YFP" "VIT" ...
 $ Round_Played       : int  351 342 372 299 230 203 272 217 203 240 ...
 $ Rating             : num  1.3 1.29 1.27 1.26 1.25 1.24 1.23 1.23 1.21 1.21 ...
 $ Avg_Combat_Score   : num  237 277 247 234 256 ...
 $ Kill_Death_Ratio   : num  1.49 1.4 1.41 1.46 1.24 1.32 1.4 1.24 1.42 1.23 ...
 $ KAST_Percentage    : num  76 70 74 76 76 80 76 75 77 71 ...
 $ Avg_Damage_Per_Round: num  156 175 162 152 168 ...
 $ Kill_Per_Round     : num  0.86 0.99 0.89 0.84 0.9 0.82 0.92 0.89 0.99 0.77 ...
 $ Assist_Per_Round   : num  0.22 0.14 0.16 0.22 0.44 0.37 0.23 0.24 0.2 0.3 ...
 $ FirstKill_Per_Round : num  0.1 0.18 0.15 0.13 0.11 0.08 0.18 0.13 0.17 0.08 ...
 $ FirstDeath_Per_Round: num  0.07 0.13 0.07 0.1 0.08 0.08 0.11 0.11 0.12 0.05 ...
 $ Headshot_Percentage : num  34 41 21 25 33 30 32 31 32 29 ...
 $ Clutch_Percentage   : num  14 14 14 19 10 21 20 13 14 23 ...
 $ Max_Kill_Single_Map : int  27 34 31 30 36 22 30 27 28 27 ...
 $ Kills              : int  303 339 331 252 207 166 250 194 200 185 ...
 $ Deaths             : int  203 243 234 173 167 126 178 156 141 151 ...
 $ Assists            : int  77 49 61 65 102 75 62 53 40 72 ...
 $ First_Kills        : int  35 60 54 39 25 17 49 28 34 19 ...
 $ First_Deaths       : int  24 43 27 30 18 17 30 23 25 13 ...
```

So, there are 21 variables and two categorical variables.

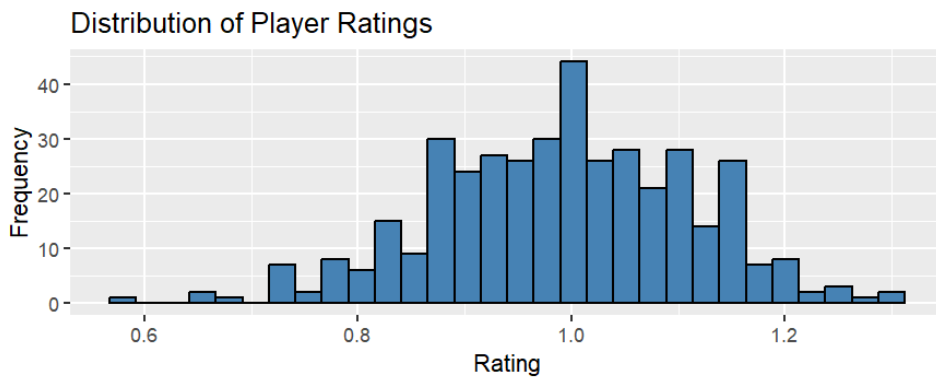
```
> summary(vlr_df)
      Rank     Player_Name            Team            Round_Played       Rating       Avg_Combat_Score Kill_Death_Ratio KAST_Percentage
 Min.   :  1   Length:417         Length:417         Min.   :200.0   Min.   :0.5800   Min.   :132.5    Min.   :0.590    Min.   :57.00
 1st Qu.:105   Class :character   Class :character   1st Qu.:242.0   1st Qu.:0.9100   1st Qu.:182.0    1st Qu.:0.900    1st Qu.:68.00
 Median :209   Mode  :character   Mode  :character   Median :297.0   Median :0.9900   Median :198.9    Median :0.990    Median :71.00
 Mean   :209                                         Mean   :315.5   Mean   :0.9921   Mean   :199.7    Mean   :1.003    Mean   :70.56
 3rd Qu.:313                                         3rd Qu.:358.0   3rd Qu.:1.0800   3rd Qu.:216.3    3rd Qu.:1.120    3rd Qu.:73.00
 Max.   :417                                         Max.   :902.0   Max.   :1.3000   Max.   :277.3    Max.   :1.490    Max.   :83.00
                                                                     NA's   :19                                         NA's   :19
 Avg_Damage_Per_Round Kill_Per_Round   Assist_Per_Round FirstKill_Per_Round FirstDeath_Per_Round Headshot_Percentage Clutch_Percentage
 Min.   : 92.7        Min.   :0.4400   Min.   :0.0800   Min.   :0.0300      Min.   :0.0300       Min.   :14.00       Min.   : 3.00
 1st Qu.:120.2        1st Qu.:0.6400   1st Qu.:0.2000   1st Qu.:0.0600      1st Qu.:0.0700       1st Qu.:25.00       1st Qu.:10.00
 Median :131.2        Median :0.7000   Median :0.2500   Median :0.0900      Median :0.0900       Median :28.00       Median :14.00
 Mean   :131.0        Mean   :0.7042   Mean   :0.2617   Mean   :0.1008      Mean   :0.1001       Mean   :28.64       Mean   :14.75
 3rd Qu.:140.8        3rd Qu.:0.7700   3rd Qu.:0.3200   3rd Qu.:0.1300      3rd Qu.:0.1200       3rd Qu.:32.00       3rd Qu.:18.00
 Max.   :174.9        Max.   :0.9900   Max.   :0.5900   Max.   :0.2300      Max.   :0.2500       Max.   :45.00       Max.   :44.00
 NA's   :19                                             NA's   :19          NA's   :19           NA's   :19          NA's   :26
 Max_Kill_Single_Map     Kills           Deaths          Assists        First_Kills      First_Deaths
 Min.   :13.00       Min.   :111.0   Min.   :125.0   Min.   : 17.00   Min.   :  0.00   Min.   :  0.00
 1st Qu.:21.00       1st Qu.:167.0   1st Qu.:171.0   1st Qu.: 55.00   1st Qu.: 17.00   1st Qu.: 19.00
 Median :24.00       Median :207.0   Median :209.0   Median : 74.00   Median : 27.00   Median : 27.00
 Mean   :24.07       Mean   :223.1   Mean   :222.1   Mean   : 82.55   Mean   : 30.56   Mean   : 30.03
 3rd Qu.:27.00       3rd Qu.:259.0   3rd Qu.:247.0   3rd Qu.:102.00   3rd Qu.: 40.00   3rd Qu.: 38.00
 Max.   :42.00       Max.   :808.0   Max.   :604.0   Max.   :292.00   Max.   :195.00   Max.   :158.00
```

From summary of the data frame, we can get the information of minimum, maximum, mean and quartile values of each variable.

Performing EDA on Key Performance Stats:

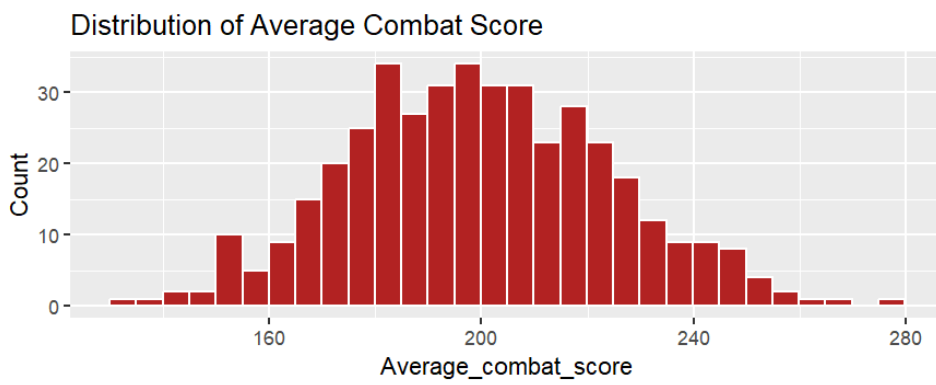
I. Distribution Analysis:

```
> ggplot(vlr_df, aes(x = Rating)) +
+   geom_histogram(fill = "steelblue", color = "black", bins = 30) +
+   labs(title = "Distribution of Player Ratings", x = "Rating", y = "Frequen
cy")
```



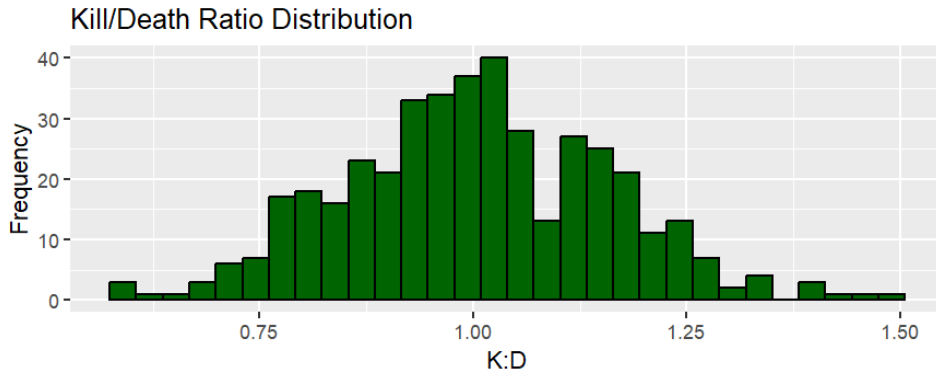
The rating distribution show how players are ranked overall. Most players cluster around average ratings but some players are at the extreme high and low ends. But this is typical for competitive ranking systems.

```
> ggplot(vlr_df, aes(x = Avg_Combat_Score)) +
+   geom_histogram(fill = "firebrick", color = "white", bins = 30) +
+   labs(title = "Distribution of Average Combat Score", x = "Average_combat_
score", y = "Count")
```



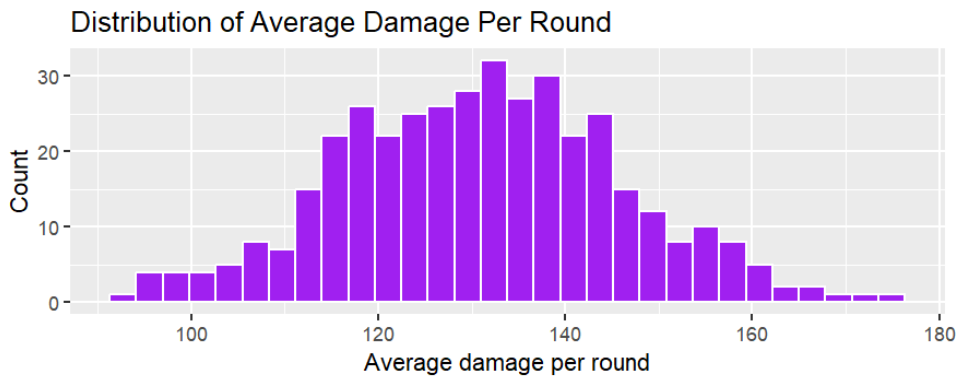
Combat score slightly shows a right-skewed distribution. Most players achieve moderate scores, with a tail expanding toward higher performers. This suggests that while average performance is common, exceptional combat score of above 240 is relatively rare.

```
> ggplot(vlr_df, aes(x = Kill_Death_Ratio)) +
+   geom_histogram( fill = "darkgreen", color = "black", bins = 30) +
+   labs(title = "Kill/Death Ratio Distribution",x = "K:D", y = "Frequency")
```



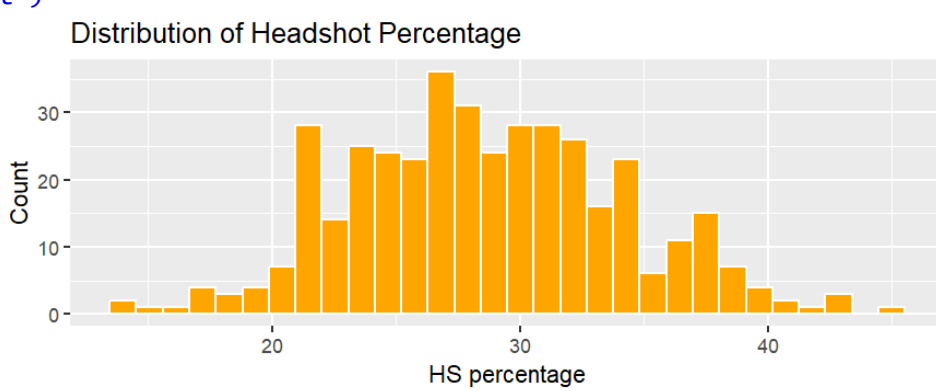
The kill death ratio distribution centers around 1.0 which indicate balance performance meaning kills equal deaths for the typical player. Players with ratios above 1.0 represent the more skilled p layer.

```
> ggplot(vlr_df, aes(x = Avg_Damage_Per_Round)) +
+   geom_histogram(fill = "purple", color = "white", bins = 30) +
+   labs(title = "Distribution of Average Damage Per Round", x = "Average dam
age per round", y = "Count")
```



The average damage per round distribution shows a relatively normal distribution where most of the player has moderate damage per round.

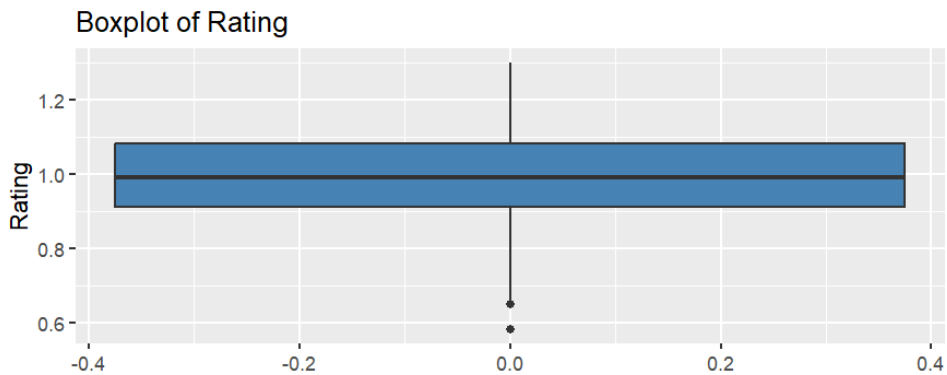
```
> ggplot(vlr_df, aes(x = Headshot_Percentage)) +
+   geom_histogram(fill = "orange", color = "white", bins = 30) +
+   labs(title = "Distribution of Headshot Percentage", x = "HS percentage",
y = "Count")
```



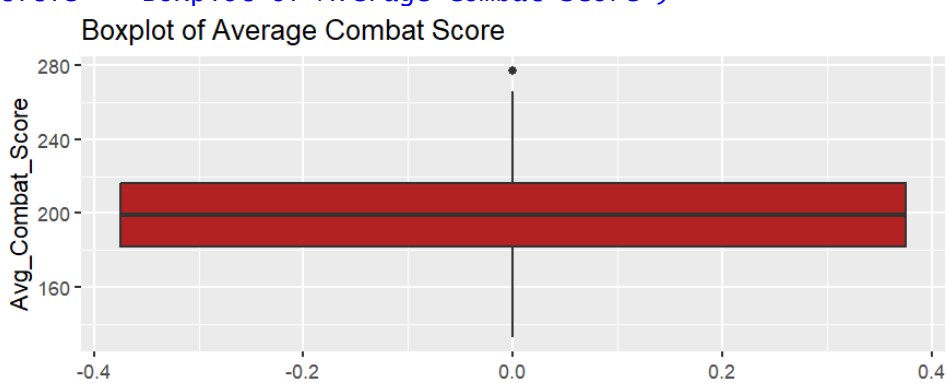
Headshot percentage shows a positively skewed distribution where most players have moderate accuracy (22-35% range). But some players with exceptional aim achieving 40%+ headshot rates are less common.

II. Detecting outlier:

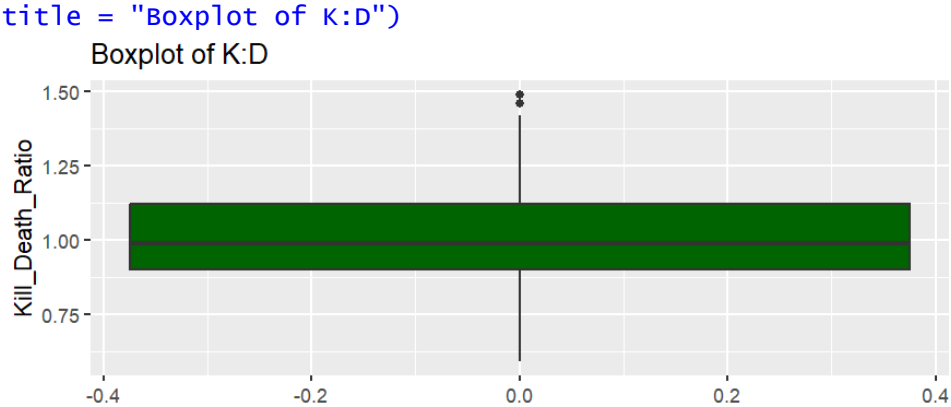
```
> ggplot(vlr_df, aes(y = Rating)) +
+   geom_boxplot(fill = "steelblue") +
+   labs(title = "Boxplot of Rating")
```




```
> ggplot(vlr_df, aes(y = Avg_Combat_Score)) +
+   geom_boxplot(fill = "firebrick") +
+   labs(title = "Boxplot of Average Combat Score")
```

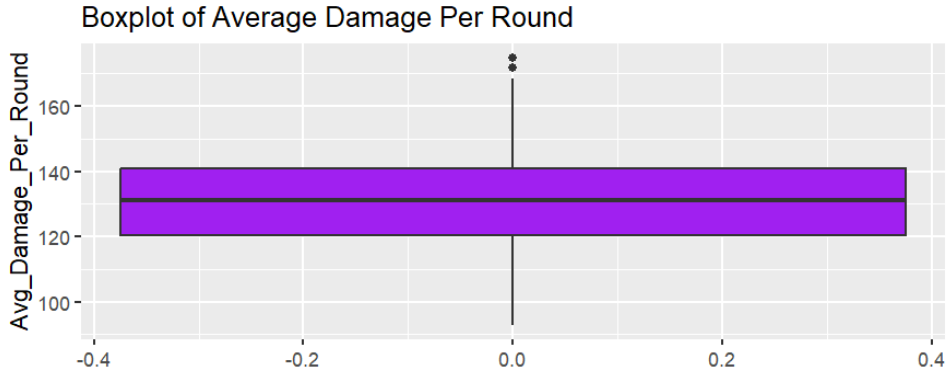



```
> ggplot(vlr_df, aes(y = Kill_Death_Ratio)) +
+   geom_boxplot(fill = "darkgreen") +
+   labs(title = "Boxplot of K:D")
```

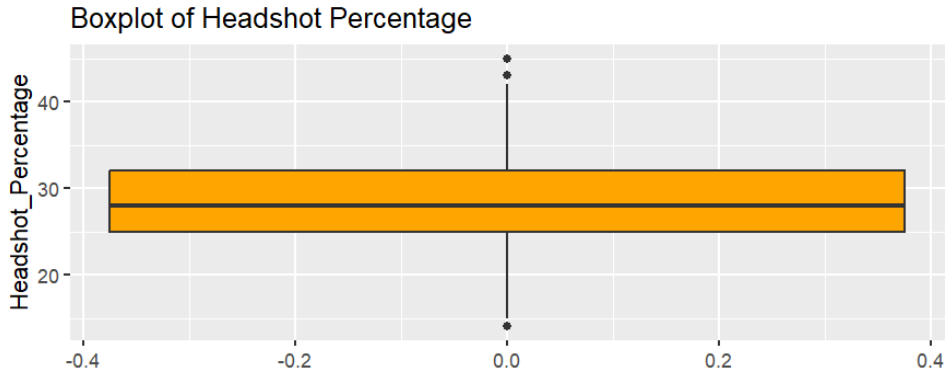



```
> ggplot(vlr_df, aes(y = Avg_Damage_Per_Round)) +
+   geom_boxplot(fill = "purple") +
```

```
+   labs(title = "Boxplot of Average Damage Per Round")
```




```
> ggplot(vlr_df, aes(y = Headshot_Percentage)) +
+   geom_boxplot(fill = "orange") +
+   labs(title = "Boxplot of Headshot Percentage")
```



III. Identifying Missing Values:

```
> colSums(is.na(vlr_df))
                Rank         Player_Name                Team        Round_Played              Rating     Avg_Combat_Score
                   0                   0                   0                   0                  19                    0
     Kill_Death_Ratio     KAST_Percentage Avg_Damage_Per_Round      Kill_Per_Round     Assist_Per_Round  FirstKill_Per_Round
                   0                  19                  19                   0                   0                   19
 FirstDeath_Per_Round  Headshot_Percentage    Clutch_Percentage  Max_Kill_Single_Map               Kills               Deaths
                  19                  19                  26                   0                   0                    0
              Assists         First_Kills        First_Deaths
                   0                   0                   0
```

So, there is presence of null values in 7 columns.

4. Data Preprocessing

Handling Missing Values:

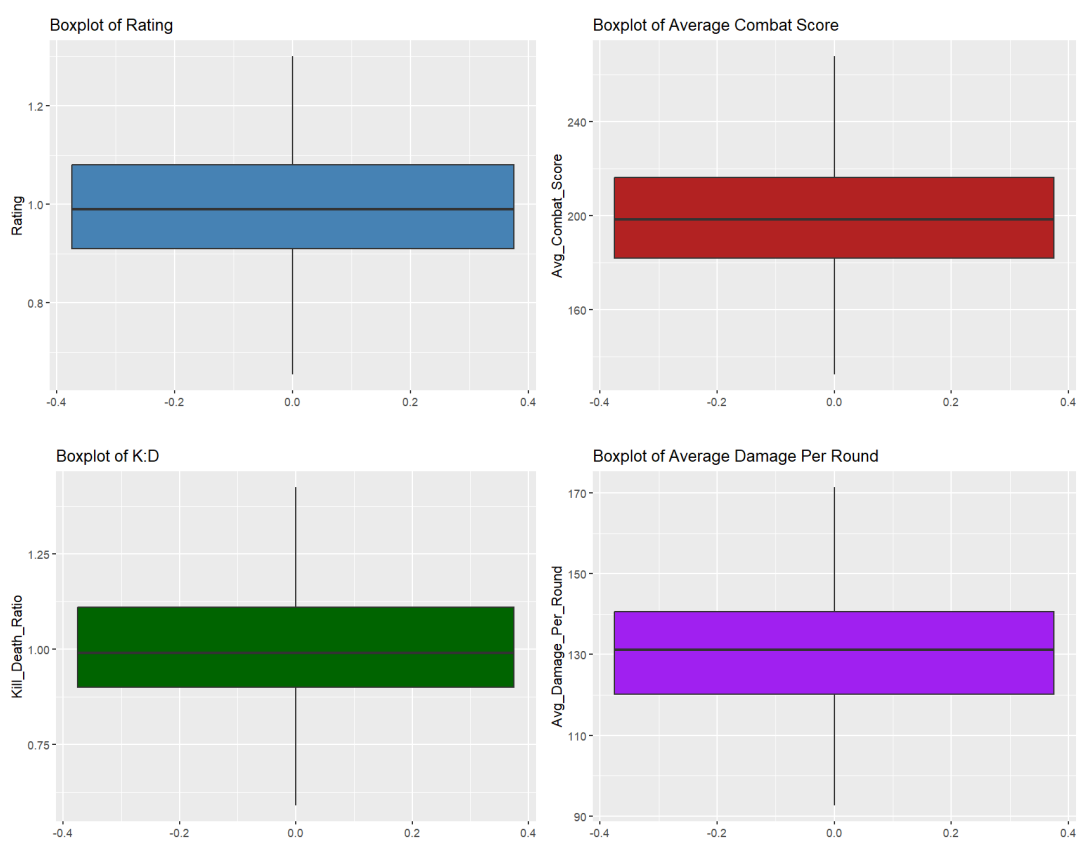
As all the columns containing null values are performance stats replacing them with mean, median will affect the overall model performance. Besides that, most rows have multiple columns missing which indicates the player being removed or killed early from the game. So, we will remove all the rows that has null values.

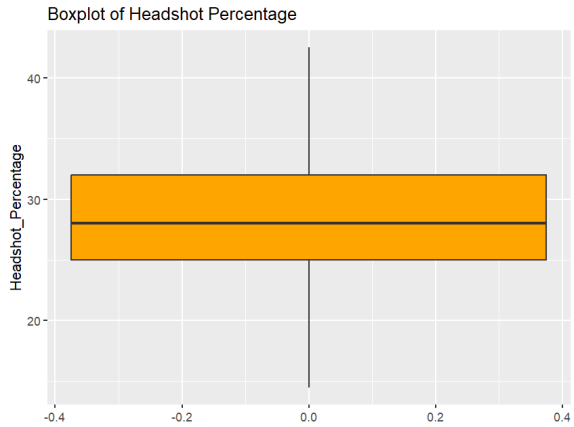
```
> colSums(is.na(vlr_df))
               Rank          Player_Name                 Team         Round_Played               Rating     Avg_Combat_Score     Kill_Death_Ratio
                  0                    0                    0                    0                   19                    0                    0
     KAST_Percentage Avg_Damage_Per_Round       Kill_Per_Round     Assist_Per_Round  FirstKill_Per_Round FirstDeath_Per_Round  Headshot_Percentage
                 19                   19                    0                    0                   19                   19                   19
   Clutch_Percentage  Max_Kill_Single_Map                Kills               Deaths              Assists          First_Kills         First_Deaths
                 26                    0                    0                    0                    0                    0                    0
> vlr_df<- na.omit(vlr_df)
> colSums(is.na(vlr_df))
               Rank          Player_Name                 Team         Round_Played               Rating     Avg_Combat_Score     Kill_Death_Ratio
                  0                    0                    0                    0                    0                    0                    0
     KAST_Percentage Avg_Damage_Per_Round       Kill_Per_Round     Assist_Per_Round  FirstKill_Per_Round FirstDeath_Per_Round  Headshot_Percentage
                  0                    0                    0                    0                    0                    0                    0
   Clutch_Percentage  Max_Kill_Single_Map                Kills               Deaths              Assists          First_Kills         First_Deaths
                  0                    0                    0                    0                    0                    0                    0
```

Handling Outliers:

```
> remove_outliers <- function(x) {
+   Q1 <- quantile(x, 0.25, na.rm = TRUE)
+   Q3 <- quantile(x, 0.75, na.rm = TRUE)
+   IQR <- Q3 - Q1
+   lower <- Q1 - 1.5 * IQR
+   upper <- Q3 + 1.5 * IQR
+   x[x < lower] <- lower
+   x[x > upper] <- upper
+   return(x)
+ }
> handle_outlier <- c("Rating", "Avg_Combat_Score", "Kill_Death_Ratio",
"Avg_Damage_Per_Round", "Headshot_Percentage")
> vlr_df[handle_outlier] <- lapply(vlr_df[handle_outlier], remove_outliers)
```

Visualization After Removing Outlier:





Handling Duplicated Value:

```
> sum(duplicated(vlr_df))
[1] 0
```

Feature Engineering:

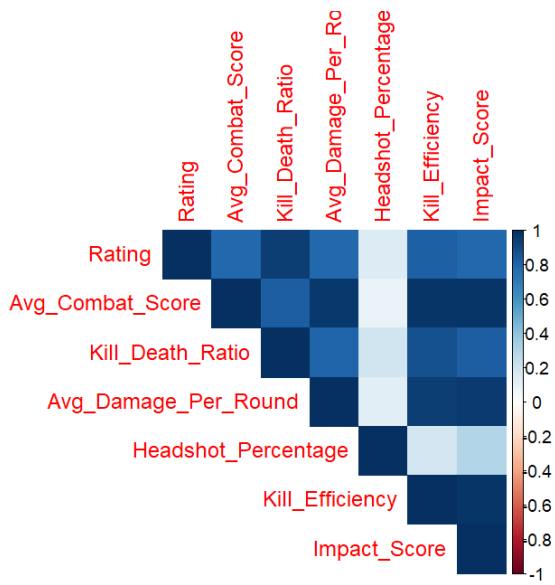
Feature engineering improves the dataset by creating new meaningful variables that improve model performance. These features have multiple performance aspects which are better than individual metrics. So, we created two new performance variables- Kill_Efficiency which is kills per round; Impact_Score which is the average of avg_combat_score, kill_death_ratio, headshot_percentage.

```
> vlr_df$Kill_Efficiency <- vlr_df$Kills / vlr_df$Round_Played
> head(vlr_df$Kill_Efficiency)
[1] 0.8632479 0.9912281 0.8897849 0.8428094 0.9000000 0.8177340
> vlr_df$Impact_Score <- (vlr_df$Avg_Combat_Score +vlr_df$Kill_Death_Ratio +vlr_df
$Headshot_Percentage) / 3
> head(vlr_df$Impact_Score)
[1]  90.77500 103.33333  89.80333  86.97500  96.64667  86.44000
```

Feature Selection:

Feature selection reduces model complexity. Since the goal is to predict rating, features with strong correlation(r>=0.5) to rating are selected.

```
> corr_features <- vlr_df %>%
+   select(Rating, Avg_Combat_Score, Kill_Death_Ratio,
+          Avg_Damage_Per_Round, Headshot_Percentage,
+          Kill_Efficiency, Impact_Score)
> cor_matrix <- cor(corr_features)
> corrplot(cor_matrix, method = "color", type = "upper")
```




```
> cor_rating <- cor_matrix["Rating", ]
> cor_rating
              Rating     Avg_Combat_Score     Kill_Death_Ratio Avg_Damage_Per_Round
           1.0000000            0.7833492            0.9451788            0.7830111
 Headshot_Percentage      Kill_Efficiency         Impact_Score
           0.1441583            0.8147013            0.7828616
```

So, the selected features are Avg_Combat_Score, Kill_Death_Ratio, Avg_Damage_Per_Round, Kill_Efficiency, Impact_Score with r = 0.78, 0.95, 0.78, 0.81, 0.78 correspondingly.

```
> selected_features <- c("Avg_Combat_Score","Kill_Death_Ratio","Avg_Damage_Pe
r_Round", "Kill_Efficiency","Impact_Score")
```

Encoding Categorical:

Since team names are categorical and regression models require numeric inputs, label encoding is applied. Label encoding maintains categorical identity without increasing dimensionality. So, it will be suitable as team is not the primary predictor.

```
> length(unique(vlr_df$Team))
[1] 161
> vlr_df$Team_Encoded <- as.numeric(as.factor(vlr_df$Team))
> head(vlr_df$Team_Encoded)
[1] 121  46 158 147 109 106
```

Scaling:

Scaling prevents features with larger values to dominate model behavior, especially in regression models.

```
> numeric_features <- c("Rating", "Avg_Combat_Score", "Kill_Death_Ratio",
+                       "Avg_Damage_Per_Round", "Headshot_Percentage",
+                       "Kill_Efficiency", "Impact_Score")
> vlr_df[numeric_features] <- scale(vlr_df[numeric_features])
```

```
> head(vlr_df[numeric_features])
    Rating Avg_Combat_Score Kill_Death_Ratio Avg_Damage_Per_Round Headshot_Percentage Kill_Efficiency Impact_Score
1 2.608742         1.502396         2.719507             1.686985           0.9879308        1.704117     1.653102
2 2.523407         2.733391         2.558616             2.697605           2.2910473        3.070383     3.094849
3 2.352735         1.907382         2.622973             2.047206          -1.4321426        1.987416     1.541551
4 2.267399         1.406162         2.719507             1.373459          -0.6875046        1.485923     1.216847
5 2.182064         2.256230         1.528915             2.494147           0.8017713        2.096468     2.327193
6 2.096728         1.145528         2.043766             1.086617           0.2432928        1.218229     1.155427
```

Final Dataset for Modeling:

```
> final_data_valorant <- vlr_df %>%
+   select(Rating, all_of(selected_features), Team_Encoded)
> str(final_data_valorant)
'data.frame':   391 obs. of  7 variables:
 $ Rating              : num  2.61 2.52 2.35 2.27 2.18 ...
 $ Avg_Combat_Score    : num  1.5 2.73 1.91 1.41 2.26 ...
 $ Kill_Death_Ratio    : num  2.72 2.56 2.62 2.72 1.53 ...
 $ Avg_Damage_Per_Round: num  1.69 2.7 2.05 1.37 2.49 ...
 $ Kill_Efficiency     : num  1.7 3.07 1.99 1.49 2.1 ...
 $ Impact_Score        : num  1.65 3.09 1.54 1.22 2.33 ...
 $ Team_Encoded        : num  121 46 158 147 109 106 61 72 159 154 ...
 - attr(*, "na.action")= 'omit' Named int [1:26] 271 273 305 334 342 369 379 399 400 401
...
  ..- attr(*, "names")= chr [1:26] "271" "273" "305" "334" ...
> summary(final_data_valorant)
     Rating         Avg_Combat_Score   Kill_Death_Ratio   Avg_Damage_Per_Round
 Min.   :-2.89541   Min.   :-2.68379   Min.   :-2.65425   Min.   :-2.5489
 1st Qu.:-0.71935   1st Qu.:-0.70497   1st Qu.:-0.65920   1st Qu.:-0.7212
 Median :-0.03667   Median :-0.04136   Median :-0.07999   Median : 0.0193
 Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.0000
 3rd Qu.: 0.73136   3rd Qu.: 0.67037   3rd Qu.: 0.69228   3rd Qu.: 0.6463
 Max.   : 2.60874   Max.   : 2.73339   Max.   : 2.71951   Max.   : 2.6976
 Kill_Efficiency      Impact_Score       Team_Encoded
 Min.   :-2.859173   Min.   :-2.49572   Min.   :  1.00
 1st Qu.:-0.709683   1st Qu.:-0.69962   1st Qu.: 34.00
 Median :-0.004216   Median : 0.01657   Median : 72.00
 Mean   : 0.000000   Mean   : 0.00000   Mean   : 75.26
 3rd Qu.: 0.696203   3rd Qu.: 0.67421   3rd Qu.:118.00
 Max.   : 3.070383   Max.   : 3.09485   Max.   :161.00
```

5. Modeling and Analysis

Model Selection:

The objective of this modeling step is to build a regression model that predicts a professional Valorant player's overall rating based on key performance indicators such as combat score, kill/death ratio, damage per round, efficiency and impact score.

Multiple Linear Regression is used because:

- The target variable is continuous
- It allows interpretation of feature impact
- It aligns with correlation-based feature selection

Final Dataset Used for Modeling:

Input Features-

- Avg_Combat_Score

- Kill_Death_Ratio
- Avg_Damage_Per_Round
- Kill_Efficiency
- Impact_Score
- Team_Encoded

Target Variable

- Rating

Process of Training, Testing and Validation:

I. Splitting Data into Training and Testing:

The dataset split into 70% training data and 30% testing data

```
> set.seed(123)
> train_index <- sample(1:nrow(final_data_valorant), 0.7 * nrow(final_data_va
lorant))
> train_data <- final_data_valorant[train_index, ]
> test_data  <- final_data_valorant[-train_index, ]
```

II. Train the Model:

```
> model <- lm(Rating ~ Avg_Combat_Score +
+               Kill_Death_Ratio +
+               Avg_Damage_Per_Round +
+               Kill_Efficiency +
+               Impact_Score +
+               Team_Encoded,
+             data = train_data)
> summary(model)

Call:
lm(formula = Rating ~ Avg_Combat_Score + Kill_Death_Ratio + Avg_Damage_Per_Round +
    Kill_Efficiency + Impact_Score + Team_Encoded, data = train_data)

Residuals:
    Min      1Q  Median      3Q     Max
-0.7871 -0.2174 -0.0088  0.2211  0.7921

Coefficients:
                       Estimate Std. Error t value Pr(>|t|)
(Intercept)           0.0370620  0.0338110   1.096  0.27400
Avg_Combat_Score      0.1740606  0.1229418   1.416  0.15800
Kill_Death_Ratio      1.0438739  0.0397285  26.275  < 2e-16 ***
Avg_Damage_Per_Round  0.2437827  0.0751245   3.245  0.00132 **
Kill_Efficiency      -0.5049646  0.1099904  -4.591 6.81e-06 ***
Impact_Score          0.0065509  0.0975004   0.067  0.94648
Team_Encoded         -0.0006380  0.0003855  -1.655  0.09912 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.2985 on 266 degrees of freedom
Multiple R-squared:  0.9125,    Adjusted R-squared:  0.9105
F-statistic: 462.3 on 6 and 266 DF,  p-value: < 2.2e-16
```

III. Predict on Test Data:

```
> predicted_rating <- predict(model, test_data)
> head(test_data["Rating"])
     Rating
1  2.608742
3  2.352735
6  2.096728
8  2.011392
12 1.670049
15 1.670049
> head(predicted_rating)
       1        3        6        8       12       15
2.621760 2.511904 1.959562 1.612129 1.264766 1.009412
```

The trained model is applied to the test dataset to predict player ratings. Comparing predicted values with actual ratings, we can see that predictions are very close.

IV. Model Performance Metric Used:

Metric	Purpose
RMSE	Measures prediction error
MAE	Average absolute error
R^2	Model explanatory power

6. Result and Interpretation

Model Performance Evaluation:

```
> RMSE <- sqrt(mean((test_data$Rating - predicted_rating)^2))
> MAE  <- mean(abs(test_data$Rating - predicted_rating))
> R2   <- cor(test_data$Rating, predicted_rating)^2
> RMSE
[1] 0.3194772
> MAE
[1] 0.2571485
> R2
[1] 0.8988726
```

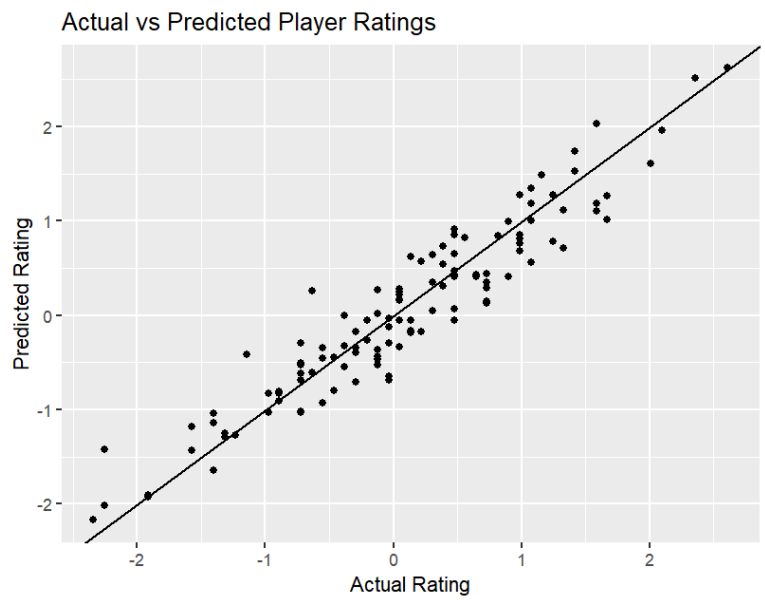
Evaluation:

- Lower RMSE and MAE indicate better prediction accuracy.
- R^2 close to 1 suggests strong predictive performance.

So, the model shows effective learning from player statistics.

Result Visualization:

```
> ggplot(data.frame(Actual = test_data$Rating,
+                   Predicted = predicted_rating),
+        aes(x = Actual, y = Predicted)) +
+   geom_point() +
+   geom_abline(intercept = 0, slope = 1) +
+   labs(title = "Actual vs Predicted Player Ratings",
+        x = "Actual Rating",
+        y = "Predicted Rating")
```

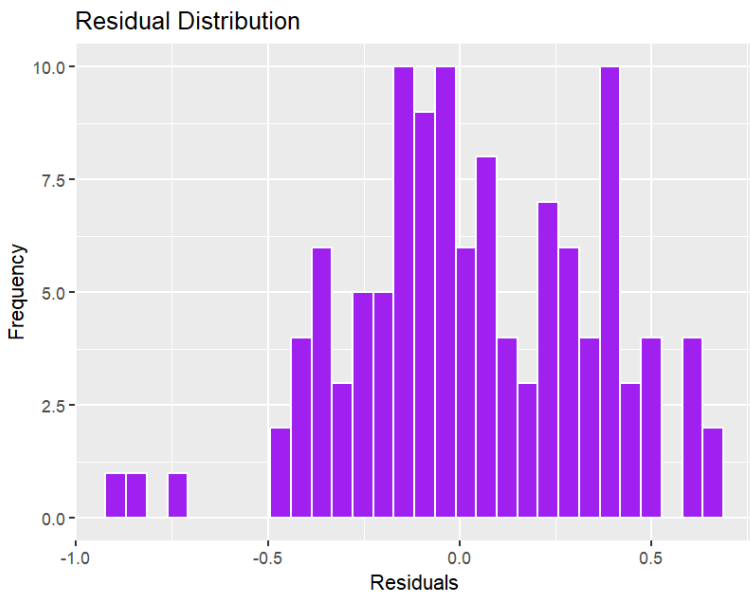


This plot demonstrates strong model performance as points clustering tightly around the diagonal line. Which means that predicted ratings closely match actual ratings across the entire range.

Residual Analysis:

Residual analysis checks whether model assumptions are satisfied.

```
> residuals <- test_data$Rating - predicted_rating
> ggplot(data.frame(Residuals = residuals),
+        aes(x = Residuals)) +
+   geom_histogram(fill = "purple", color = "white", bins = 30) +
+   labs(title = "Residual Distribution",
+        x = "Residuals",
+        y = "Frequency")
```



Feature Importance Interpretation:

Features with larger absolute coefficients have greater influence on player rating.

```
> coef(summary(model))
                         Estimate   Std. Error     t value     Pr(>|t|)
(Intercept)           0.0370620241 0.0338110358  1.09615169 2.740038e-01
Avg_Combat_Score      0.1740606372 0.1229418256  1.41579675 1.580048e-01
Kill_Death_Ratio      1.0438739019 0.0397284826 26.27520190 6.986603e-76
Avg_Damage_Per_Round  0.2437826920 0.0751244697  3.24505042 1.324600e-03
Kill_Efficiency      -0.5049645927 0.1099903642 -4.59098937 6.809824e-06
Impact_Score          0.0065508877 0.0975004457  0.06718828 9.464823e-01
Team_Encoded         -0.0006380222 0.0003855341 -1.65490474 9.912319e-02
```

In a regression model:

- A positive coefficient means the feature increases player rating.
- A negative coefficient means the feature decreases player rating.
- A larger absolute value means stronger impact.
- p-value < 0.05 means the feature is statistically significant.

Feature	Coefficient	Impact	P-Value	Significance
Avg_Combat_Score	+0.174	Low positive	0.158	Low
Kill_Death_Ratio	+1.044	High positive	<2e-16	High
Avg_Damage_Per_Round	+0.244	Moderate positive	0.00132	Medium
Kill_Efficiency	-0.505	Moderate negative	6.81e-06	High
Impact_Score	+0.0065	Low positive	0.946	Low
Team_Encoded	-0.0006	Low negative	0.099	Low

So, kill_death_ratio feature has the strongest impact on player rating.

Overall Model Performance Interpretation:

- R^2 (Train): 0.9125
- R^2 (Test): 0.8989
- RMSE: 0.319
- MAE: 0.257

The model explains almost 90% of the variation in player ratings and predicts ratings with relatively low errors. This indicates a strong and reliable regression model.

7. Conclusion and Future Work

This study aimed to analyze and predict professional Valorant player ratings using performance statistics collected through web scraping. After preprocessing and exploratory analysis, a multiple linear regression model was developed to predict player ratings. The results show that Kill/Death Ratio is the most influential factor, followed by average damage per round. These findings confirm that individual combat efficiency plays a critical role in determining player performance. The model achieved strong predictive accuracy, explaining nearly 90% of the variance in player ratings.

Despite strong results, the study has several limitations. The dataset was collected from a single statistics website and does not account for contextual factors such as map type, player roles, or opponent strength.

Future work can improve this study by incorporating role-based features, map-specific performance metrics. Advanced machine learning models such as Random Forest or XGBoost could also be applied to capture non-linear relationships.